

CYBR 437: Secure Coding

Vulnerability Assessment

Instructor: Abinash Borah

Outline

- Introduction to Vulnerability Assessment
- Categories of Vulnerabilities
- Stages of Vulnerability Assessment
- Vulnerability Assessment Methodologies

Vulnerability Assessment

- Vulnerability assessment is a systematic, often automated, process of identifying, classifying, and prioritizing security weaknesses in software, hardware, and network infrastructure
- It involves scanning systems to detect vulnerabilities, analyzing risk, and reporting recommended remediations to prevent cyberattacks



- Software vulnerability assessment focuses specifically on identifying security weaknesses inside applications

Software Vulnerability Assessment

- A software vulnerability is a flaw, bug, or weakness in an application due to coding issues, design oversights, or architecture that can be exploited by threat actors
- These security gaps allow unauthorized access, data theft, or service disruption, directly impacting the confidentiality, integrity, or availability of a system
- The software vulnerability assessment process involves:
 - Identifying coding flaws
 - Detecting insecure design patterns
 - Discovering unsafe configurations
 - Evaluating third-party dependencies
 - Remediation

Categories of Software Vulnerabilities

- Software vulnerabilities can be categorized based on the root cause, the affected component, and potential impact on confidentiality, integrity, or availability
- NIST categorizes the following “Classes of Software Security Flaws and Vulnerabilities”:
(<https://www.nist.gov/itl/ssd/software-quality-group/taxonomy-software-flaws>)
- **Input Validation Error:** Input passed to an application is not properly validated, such that vulnerability can be exploited by a certain input sequence
- **Boundary Overflow:** Input exceeds an assumed boundary thereby causing vulnerability. For example, the application may run out of memory, a variable might reach its maximum value and roll over to its minimum value, etc.

Categories of Software Vulnerabilities (contd.)

- **Buffer Overflow:** A special case of Boundary Overflow, where the bounds checking on the size of input being stored in a buffer array is not performed or in error. This type of vulnerability comes with different flavors, include:
 - **Upper/Lower Bound:** Upper or lower bound is violated.
 - **Data Type:** Buffer of all data types (character, integer, floating point, wide character, pointer, unsigned character, and unsigned integer) may be overflowed. The attacker may use different technique for different data type to exploit the buffer overflow vulnerability.
 - **Memory Location:** Buffer may reside in different locations (stack, heap, data region, BSS, shared memory, etc.). The attacker may use different technique for different memory location to exploit the buffer overflow vulnerability.

Categories of Software Vulnerabilities (contd.)

- **Malformed Input:** Input passed to a procedure call in an application is not properly checked such that vulnerability can be exploited by a certain input sequence.
- **Tainted Input to Call-Out Resources**
 - **Command Injection:** Input passed to a command isn't properly sanitized before being used in a command execution call such as `system()`, `exec()` or `popen()`. This can be exploited to inject arbitrary shell commands.
 - **SQL Injection:** Input passed to SQL query of an application, mostly found within webpages with dynamic content, isn't properly sanitized before being used. This can be exploited to cause computer security breach.
- **Format String:** Hostile input passed as the format string for a variable arguments routine such as `printf`, the attacker can write arbitrary values to memory. The `%n` directive is particularly susceptible to attack.

Categories of Software Vulnerabilities (contd.)

- **Cross-Site Scripting:** A website may inadvertently include malicious HTML tags or script in a dynamically generated page based on unvalidated input from untrustworthy sources. This can be a problem when a web server does not adequately ensure that generated pages are properly encoded to prevent unintended execution of scripts, and when input is not validated to prevent malicious HTML from being presented to the user.
- **Directory Traversal:** Directory traversal is an exploit that performs malicious activities such as accessing restricted directories, executing commands and viewing data outside the normal server directory where the application content is stored. Commonly, this vulnerability occurs when the web server software fails to validate input received from browsers.
- **Exceptional Condition Handling Error:** The handling (or mishandling) of the exception by the application enables a vulnerability.

Categories of Software Vulnerabilities (contd.)

- **Symlink Problem:** When temporary file being created insecurely, it can be exploited via symlink attacks to create and overwrite arbitrary files with the privileges of the user running the affected script.
- **Insecure Access Control Application level:** Access control mechanism is faulty; bad permission.
 - **Privilege Escalations:** Unauthorized users gain escalated privileges.
 - **Improper Database Access:** Database server improperly sets the access control for its clients. The client might be able to run arbitrary code on the server.
- **Cryptographic Error Application level:** Cryptographic service is faulty.
- **Random Number:** A random number vulnerability occurs when a program uses a method of generating random numbers is predictable, e.g. `rand()`, `random()`.

Categories of Software Vulnerabilities (contd.)

- **Race Conditions:** A race condition occurs when multiple processes access and manipulate the same data concurrently, and the outcome of the execution depends on the particular order in which the access takes place. A race condition is of interest to a hacker when the race condition can be utilized to gain privileged system access.
- **Memory Leak:** A flaw in a program that prevents it from freeing up memory that it no longer needs. As a result, the program grabs more and more memory until it finally crashes because there is no more memory left.
- **Null Pointer Dereference:** Under some circumstances, a null pointer may be dereferenced during a memory allocation.
- **Backdoor/Input Sensitivity:** Some specific input triggers malicious code.

Stages of Vulnerability Assessment

1. Planning and Scoping Process Components

- Planning and scoping involve defining the assessment's goals, identifying assets, and setting boundaries to identify, rank, and address security defects
- The key steps are:
 - Determination of the assessment's goal (e.g., regulatory compliance, risk reduction) and definition of the target boundaries (applications, network, or hosts)
 - Enumeration of all software, systems, and network components to be tested
 - Prioritization of systems based on their criticality
 - Determination of the methods to be used, such as using vulnerability scanners, manual testing, etc.
 - Definition of the timeframe and limitations to minimize application impact during testing

Stages of Vulnerability Assessment (contd.)

2. Vulnerability Discovery and Scanning

- Vulnerability discovery involves manual analysis, reverse engineering, or specialized security research to discover previously unknown vulnerabilities
- Scanning involves the use of automated tools (e.g., scanners) and manual checks to scan code and infrastructure for known vulnerabilities
- Manual checks can be incorporated to supplement automated scans

Stages of Vulnerability Assessment (contd.)

3. Analysis/Evaluation

- Analysis of the findings to determine the root cause of each vulnerability
- Interpretation of the scan results to filter out false positives
- Designating severity levels (such as critical, high, medium, low, etc.) to each vulnerability based on their potential risks

Stages of Vulnerability Assessment (contd.)

4. Reporting/Documentation

- Preparation of a detailed report for stakeholders
- Documentation of the findings, potential impacts, and recommended remediation steps for the identified weaknesses

5. Remediation and Validation

- Prioritization and fixation of the identified vulnerabilities, focusing on the most critical ones first
- Re-scanning of the system to validate that the fixes have successfully removed the vulnerabilities

Vulnerability Assessment Methodologies

- Vulnerability assessment involves various complementary techniques that identify security flaws
- The primary methods for software vulnerability assessment are:
 - Static Analysis
 - Dynamic Analysis
 - Fuzz Testing
 - Software Composition Analysis
 - Penetration Testing

Static Analysis

- Static code analysis includes various types, with the main two being pattern-based and flow-based
- A simple example of static analysis is using automated tools to inspect source code for issues without running the program – these tools analyze the code's structure, logic, and data flow to detect problems early in development
- **Pattern-based static analysis** looks for code patterns that violate defined coding rules
- In addition to ensuring that code meets uniform expectations for regulatory compliance or internal initiatives, it helps prevent defects such as resource leaks, performance and security issues, logical errors, and API misuse

Static Analysis (contd.)

- **Flow-based static analysis** involves finding and analyzing the various paths that can be taken through the code
- This can happen by control (the order in which lines can be executed) and by data (the sequences in which variables and data structures can be created, used, and destroyed)
- Can expose problems that lead to critical defects, such as memory corruptions, memory access violations, null pointer dereferences, etc.
- Can also detect security issues by pointing out paths that bypass security-critical code, such as code for authentication or encryption

Dynamic Analysis

- Dynamic analysis examines software while it runs to detect issues that only appear at runtime
- It evaluates how the program behaves, interacts with memory, and responds to inputs
- A common example is unit and integration testing, where functions or components are executed with controlled inputs
- Another example is runtime memory analysis, which monitors memory allocation and usage to catch problems that only appear during execution, such as leaks, buffer overflows, and null pointer dereference

Fuzz Testing

- Fuzz testing (or fuzzing) involves providing invalid, unexpected, or random data as inputs to a program to expose coding errors and security vulnerabilities
- Helps discover bugs that traditional testing might miss, such as buffer overflows, memory leaks, and injection flaws
- The fuzzing process generally involves a few key steps:
 - *Input Generation*: A fuzzer (testing tool) generates a large volume of test cases – can be entirely random, or based on existing valid inputs or predefined specifications
 - *Execution*: The generated inputs are fed into the target application
 - *Monitoring*: The system is monitored for exceptions, such as crashes, infinite loops, resource depletion, or violations of security invariants (predefined rules about expected behavior)
 - *Reporting and Analysis*: When a failure is detected, the fuzzer logs the specific input that caused the issue, the location in the code (with white-box fuzzing), and other diagnostic information

Types of Fuzzing

Black-box Fuzzing (Dumb Fuzzers)

- Has no knowledge of the program's internal structure and relies solely on feeding random or mutated inputs to the system's external interfaces
- Simple but may be less efficient at reaching deep code paths

White-box Fuzzing (Source Code Fuzzing)

- Analyzes the program's source code or binary to guide the input generation, aiming for maximum code coverage and exploring specific execution paths
- Complex to implement but highly effective at finding deep-seated vulnerabilities

Gray-box Fuzzing

- Hybrid approach that uses feedback based on observed behavior or basic code coverage information to guide the input generation, balancing efficiency and effectiveness without full source code analysis

Software Component Analysis

- Software Component Analysis (SCA) is a process used to identify, manage, and monitor third-party and open-source components within an application
- Modern software heavily depends on external libraries (e.g., Maven packages, pip libraries) and SCA helps detect known vulnerabilities, license compliance issues, outdated dependencies, etc.
- Developers may unknowingly include vulnerable libraries that may eventually lead to security issues
- The problem is growing with massive open-source usage and a focus on rapid development
- Attackers also increasingly target the software supply chain

Penetration Testing

- Penetration testing (often called pen testing) is a structured, authorized simulation of real-world cyberattacks against software, systems, or networks to identify security vulnerabilities
- It is a key part of software vulnerability assessment, but it goes beyond scanning - it actively attempts to exploit weaknesses
- Penetration testing is an ethical hacking process where security professionals attempt to identify vulnerabilities, exploit weaknesses safely, assess potential impacts, and provide remediation guidance
- Penetration testing can also be Black-box, White-box, and Gray-box

Common Mistakes in Vulnerability Assessment

- Over-reliance on automated tools
- Ignoring application logic flaws
- Not validating the findings
- Lack of integration with the development life cycle
- Treating vulnerability assessment as a one-time activity